

# ACMEMobility

Progetto di Architetture Software a Microservizi, Università di Bologna, 2025–2026

Daniele D’Ugo  
daniele.dugo@studio.unibo.it  
Mat.: 0001186581

Franck-Steve Tcheuffa  
franck-steve.tcheuffa@studio.unibo.it  
Mat.: 0001232917

Adamou Nchare Pekassa  
adamou.ncharepekassa@studio.unibo.it  
Mat.: 0001244371

## I. INTRODUZIONE

Il servizio di noleggio ACMEMobility è stato implementato come un’architettura a microservizi, tramite *docker compose*, cercando di rispettare tutti i requisiti del progetto.

## II. AVVIO E UTILIZZO

L’intera architettura, comprendente tutti i servizi, con il codice nella cartella *src/*, può essere avviata e gestita tramite i comandi di *docker compose*; per comodità, sono presenti due script *bash* (per sistemi Linux). *start.sh* lancia l’infrastruttura:

```
1 # dalla cartella di base del repository
2 ./src/start.sh
```

*down.sh* interrompe i servizi:

```
1 ./src/down.sh
```

Prima, però, è necessario scaricare la mappa (con dati di OpenStreetMap) dell’Italia nord-orientale, disponibile al link su geofabrik: <https://download.geofabrik.de/europe/italy/nord-est-latest.osm.pbf>.

Il file dovrà essere piazzato in *service-graphhopper/data/*, con nome del file *geofabrik\_italy-nord-est-260517.osm.pbf*.

Da Linux, si può eseguire direttamente, per entrambi i passaggi, il comando:

```
1 # dalla cartella di base del repository
2 wget -O service-graphhopper/data/geofabrik_italy-nord-est-260517.osm.pbf \
3 https://download.geofabrik.de/europe/italy/nord-est-latest.osm.pbf
```

Il flusso delle attività parte sempre dall’utente, che in uno scenario reale sarebbe un’entità fisica e potrebbe interagire con ACMEMobility tramite un’applicazione per smartphone. Dunque, abbiamo realizzato uno script in *python* (*src/user/user.py*) che simula tutte le possibili azioni che possa compiere, in base agli argomenti utilizzati. Seguono degli esempi per ogni utilizzo. Usage:

```
1 # dalla cartella di base del repository
2 python src/user/user.py
3 # Usage: python user.py <action> <user_id> <vehicle_id> [station_id]
4 # Actions: scan, reserve, cancel, scan_reserved, park, lock, lock_assistance, steal
```

Scansione veicolo (con id *V-01*) per uso immediato, da parte dell’utente con id *USER-001*:

```
1 python src/user/user.py scan USER-001 V-01
```

Prenotazione veicolo per dopo:

```
1 python src/user/user.py reserve USER-001 V-01
```

Cancellazione della prenotazione precedente:

```
1 python src/user/user.py cancel USER-001 V-01
```

Scansione del veicolo precedentemente prenotato:

```
1 python src/user/user.py scan_reserved USER-001 V-01
```

Parcheggio del veicolo (simulazione dell’inserimento del veicolo nel dock della stazione con id *STATION-BOLOGNA-01*):

```
1 python src/user/user.py park USER-001 V-01 STATION-bologna-01
```

Richiesta di fine corsa e, dunque, blocco del veicolo precedentemente parcheggiato:

```
1 python src/user/user.py lock USER-001 V-01 STATION-bologna-01
```

Richiesta di assistenza a fine corsa, in caso di problemi con il blocco:

```
1 python src/user/user.py lock_assistance USER-001 V-01
```

Simulazione furto di un veicolo:

```
1 python src/user/user.py steal USER-001 V-01
```

Invia un’istruzione a Camunda per eseguire immediatamente il task di timeout per il tempo di cancellazione di una prenotazione (utile per testare l’applicazione della penalità):

```
1 python3 src/user/user.py force_timeout USER-001 V-01
```

Una volta avviata l’infrastruttura, è possibile accedere alla visualizzazione di Camunda al link <http://localhost:8080/camunda/app/cockpit/>, con username *demo* e password *demo*.

Al link <http://localhost:4000/map/>, invece, è possibile visualizzare una mappa di Bologna, con dei segnali per le stazioni ed eventuali veicoli in movimento.

I veicoli utilizzano coordinate reali e ricevono dei percorsi da un’istanza, eseguita in locale, di *GraphHopper* — che espone anche un’interfaccia web al link <http://localhost:8989/>, non particolarmente rilevante per il progetto.

Sono inoltre disponibili altri endpoint, esposti dai vari servizi per debugging. Seguono quelli più utili per il debugging.

Il servizio di ACMEMobility espone il suo database:

```
1 curl localhost/db
```

Ogni stazione espone il suo stato:

```
1 curl localhost:5001/station
2 curl localhost:5002/station
3 curl localhost:5003/station
4 curl localhost:5004/station
5 curl localhost:5005/station
```

### III. WORKFLOW E ARTEFATTI

Lo sviluppo del progetto è partito con la modellazione di una coreografia in BPMN con scopo documentativo. Usandola come schema è stato così possibile suddividere, tra i membri del gruppo, i compiti di realizzare ulteriori modelli o i singoli servizi.

La coreografia è stata poi gradualmente raffinata, fino a diventare la attuale coreografia a scopo documentativo (file *docs/diagrams/diagram-collaboration-full.bpmn*). Essa raffigura infatti tutti i partecipanti e i loro task, per rendere l’idea di quali feature debba implementare ogni servizio.

Per Camunda è stato modellato un altro diagramma di collaborazione eseguibile

(file *src/camunda-resources/diagram-collaboration-camunda.bpmn*), con la principale differenza che non implementa nel dettaglio tutti i partecipanti, ma solo il servizio principale di ACMEMobility, che era richiesto che fosse gestito tramite BPMS. Esiste anche un altro diagramma eseguibile di debugging (file *src/camunda-resources-debug/diagram-collaboration-camunda-debug.bpmn*), che presenta degli *user task* prima di ogni end task, per semplificare l’osservazione manuale dei processi: poiché i due diagrammi

eseguibili sono per il resto identici (hanno le stesse variabili, etc.), si possono scambiare direttamente dal file di *docker-compose* (*docker-compose.template.yml*), selezionando la cartella opportuna come volume per il servizio di *Camunda*.

Successivamente, sono stati realizzati un diagramma di documentazione della SOA in UML, con profilo TinySOA (file *docs/diagrams/diagram-SOA.uml* e *docs/diagrams/diagram-SOA-UML.uml*), e una coreografia con sintassi formale, necessaria per le proiezioni della coreografia sui ruoli e per l'analisi di *connectedness*.

I servizi implementati, invece, sono:

- ACME (*src/service-acme-worker/*): il worker che implementa i service task del BPMS per il core di ACMEMobility, scritto in *python* con l'apposito pacchetto per *Camunda*;
- bank (*src/service-bank/*): la banca, in *jolie*;
- Fleet Management (*src/service-fleet-management/*): implementato come un router, in *Nginx*, così che appaia esso stesso come un'architettura a microservizi;
  - Fleet Management Battery (*src/service-fleet-battery/*): microservizio per gestire la batteria dei veicoli, in *python* con *Flask*;
  - Fleet Management Tracking (*src/service-fleet-tracking/*): microservizio per gestire posizione e informazioni aggiuntive dei veicoli, in *python* con *Flask*;
  - Fleet Management Visualizer (*src/service-fleet-visualizer/*): semplice file *html* per visualizzare una mappa con spostamenti in tempo reale (implementata tramite la libreria javascript *leaflet*), servito come pagina statica da *Nginx*;
- graphhopper (*src/service-graphhopper/*): routing service per fornire dei percorsi reali ai veicoli, date posizione di partenza e destinazione; configurato con il container *docker* già esistente;
- station (*src/service-stations/*): implementazione di una singola stazione, in *python* con *Flask*;
- vehicle (*src/service-vehicle/*): implementazione di un singolo veicolo, in *python* con *Flask*.

Tutti i servizi sono gestiti tramite un'architettura con *docker-compose*, che si occupa anche di istanziare molteplici veicoli e stazioni, con la stessa immagine e implementazione. Il file è scritto come template in *src/docker-compose.template.yml*; uno script *bash* (*src/bash-scripts/compile-dockercompose.sh*, già invocato da *start.sh*) lo lavora, per esempio sostituendo delle variabili con quelle in *src/.env*, generando quello finale (*src/docker-compose.generated.yml*). In questo modo, i veicoli e le stazioni istanziati sono ben riconoscibili poiché generati con identificativi definiti tramite variabili di ambiente.

Per la coreografia in *Camunda*, come già accennato, tutti i service task sono implementati tramite un worker in *python*. Solo delle operazioni banali, principalmente le condizioni di scelta per i gateway esclusivi, sono implementate come espressioni *inline*, con sintassi *JUEL* (*Java Unified Expression Language*), utilizzando delle variabili globali che vengono configurate dallo stesso worker. L'implementazione esterna, infatti, fornisce grandi vantaggi nello sviluppo, in termini di gestibilità, mantenibilità e leggibilità del codice, possibilità di debugging e gestione coerente e ordinata dei file.

Inoltre, tutti i servizi usano *API REST* e comunicano tramite *http*, ad eccezione della banca, che usa *SOAP* come richiesto.

In *docs/implementation/* sono presenti le documentazioni sia per le API dei servizi (incluso ACMEMobility, a cui lo script *user.py* invia le richieste), sia per le variabili globali di *Camunda* (*bpms-variables.md*).

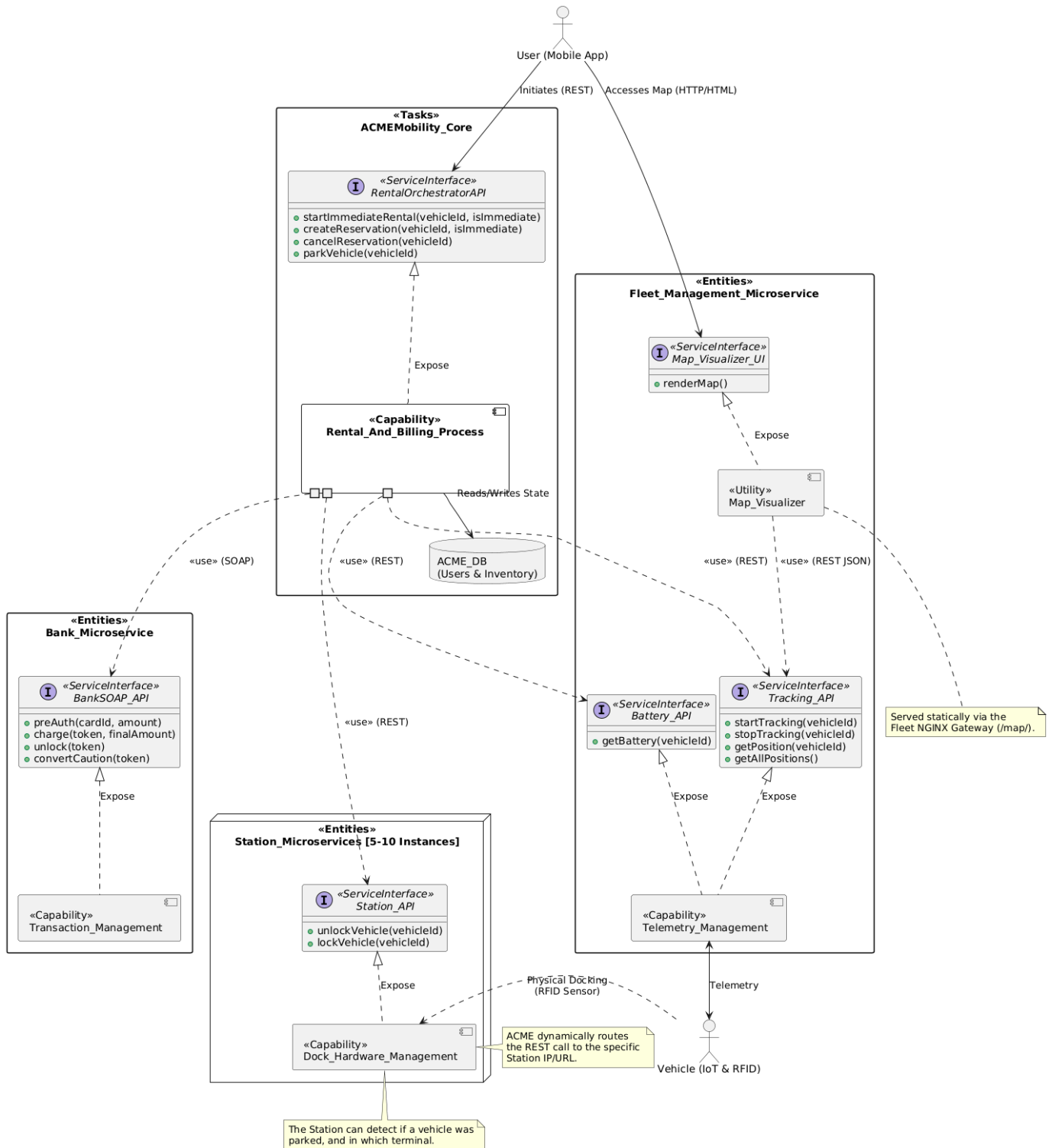
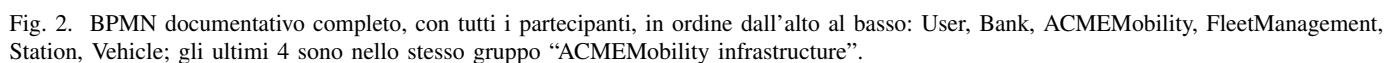


Fig. 1. Diagramma SOA in UML.

#### IV. IMPLEMENTAZIONE

In questa sezione presentiamo il modo in cui vengono eseguiti i processi (e quindi la coreografia), insieme a commenti e motivazioni per le scelte di design.

Mostreremo anche il diagramma BPMN documentativo intero e poi a pezzi: quello integrale è visionabile anche nel repository del progetto (in *docs/diagrams/*). Non mostriamo qui il diagramma eseguibile



1) *Prospettiva user*: Come già accennato, il primo processo a partire è quello dell'**utente**, che va poi ad avviare a catena gli altri, con dei *message start event*. Nella **fase 1**, la **prenotazione**, l'utente parte con una scelta: scansionare il codice QR di un veicolo subito oppure prenotarne uno; nel secondo caso, può in

seguito scansionare il veicolo per far partire la corsa, oppure cancellare la prenotazione e quindi terminare il suo processo. In entrambi i casi, dopo essere giunto con successo alla scansione, inizia la **fase 2**, il **tragitto**, estremamente semplice dal punto di vista dell'utente, meno dal punto di vista di ACMEMobility. Infine, nella **fase 3**, la **conclusione**, l'utente deve interrompere la corsa e attendere la notifica di conferma da parte dell'applicazione. L'interruzione della corsa è in realtà ulteriormente articolata: bisogna prima inserire il veicolo in un terminale della stazione (chiamato anche "dock") e, successivamente, richiedere il blocco del veicolo ad ACMEMobility, tramite l'applicazione. In caso di problemi nel parcheggiare, si possono ripetere più volte le due operazioni di parcheggio e blocco finché non si abbia successo, oppure contattare l'assistenza (interrompendo il processo della coreografia): in questo modo, un addetto verrà inviato a controllare personalmente il veicolo, mentre per l'utente ACMEMobility procederà con l'addebito, in modo che possa andarsene e non debba rimanere ad aspettare. Qui, per gestire il fatto che il veicolo venga lasciato incustodito, assumiamo che, nel mondo reale, esso abbia qualche sistema di bloccaggio automatico della guida, o che magari funzioni solo quando un utente correttamente autorizzato (attraverso la **fase 1**) sia nelle vicinanze (il che potrebbe essere confermato dalla vicinanza del suo telefono, per esempio via *Bluetooth*).

Nel parcheggio, mentre il blocco del veicolo passa attraverso ACMEMobility (attraverso un *message intermediate throw event*), il precedente inserimento è modellato direttamente come un messaggio verso la stazione: nel mondo reale, infatti, immaginiamo che veicolo e terminale possano essere configurati con sensori per comunicazione a breve raggio (per esempio, per riconoscere un tag *RFID* o *NFC* sul veicolo), grazie ai quali la stazione può identificare se ci sia un veicolo inserito, prima di bloccarlo. D'altra parte, ACME, all'arrivo della notifica di parcheggio, procede, in ordine, a chiedere alla stazione il bloccaggio, effettuare l'addebito di fine corsa e confermare l'operazione all'utente con una notifica. La separazione in inserimento e bloccaggio è stata fatta per rendere più solido il design di questa parte: in questo modo, l'applicazione dell'utente deve solo comunicare gli identificativo di stazione e veicolo, e la stazione saprà automaticamente quale terminale bloccare; la comunicazione a breve raggio, inoltre, permette a veicolo e stazione di scambiare eventuali informazioni di conferma.

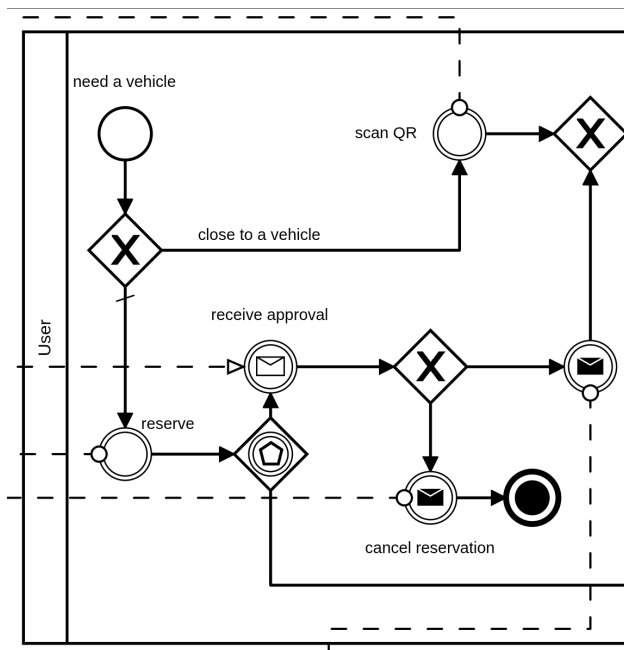


Fig. 3. BPMN documentativo per User, fase 1

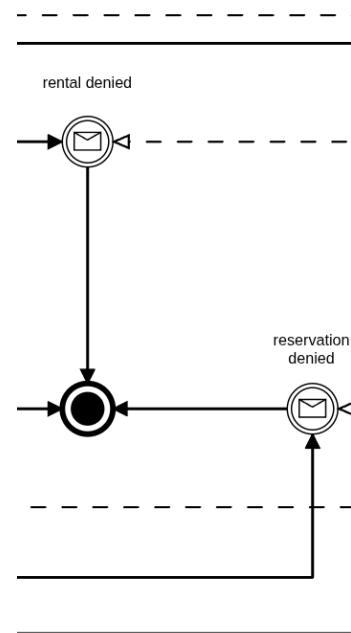


Fig. 4. BPMN documentativo per User, fase 1, casi di errore

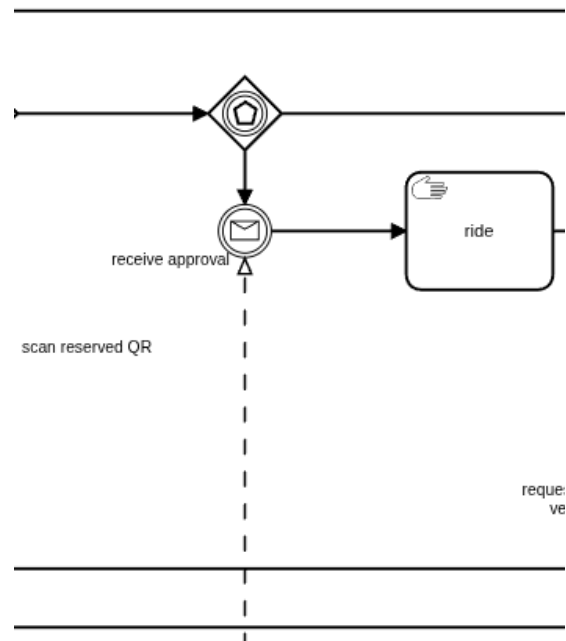


Fig. 5. BPMN documentativo per User, fase 2

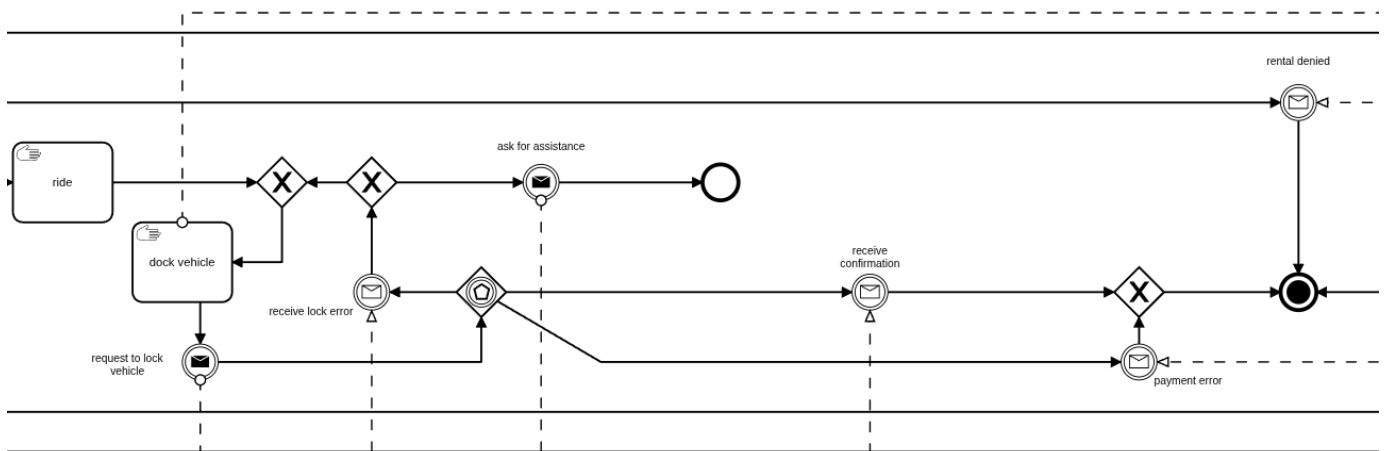


Fig. 6. BPMN documentativo per User, fase 3

2) *Prospettiva di ACMEMobility*: Dal punto di vista di **ACMEMobility**, la coreografia appena descritta è più complessa e articolata.

**Fase 1 (prenotazione)** — un processo di ACME può partire solo da un messaggio, ricevuto dall'utente, di noleggio immediato o prenotazione: i due possibili processi si uniscono subito (con un gateway), ma l'informazione sul tipo di noleggio rimane memorizzata, in Camunda, come variabile globale (maggiori dettagli in seguito).

Subito dopo, si effettua il blocco della cauzione, richiedendolo alla banca. Per di più, solamente in caso di prenotazione, ACME effettua alcune operazioni interne, come aggiornare i database e salvarne un timestamp, e si mette in attesa di uno di tre possibili eventi: inizio della corsa (con scansione del veicolo da parte dell'utente), cancellazione della prenotazione o scadenza del tempo (30 minuti). In caso di annullamento della corsa (per richiesta o per scadenza del tempo), dunque, sblocca la cauzione o la converte in addebito in base al tempo trascorso, sempre tramite la banca.

Se e quando il processo arriva alla **fase 2 (tragitto)**, ACME si occupa di tenere monitorato il veicolo.



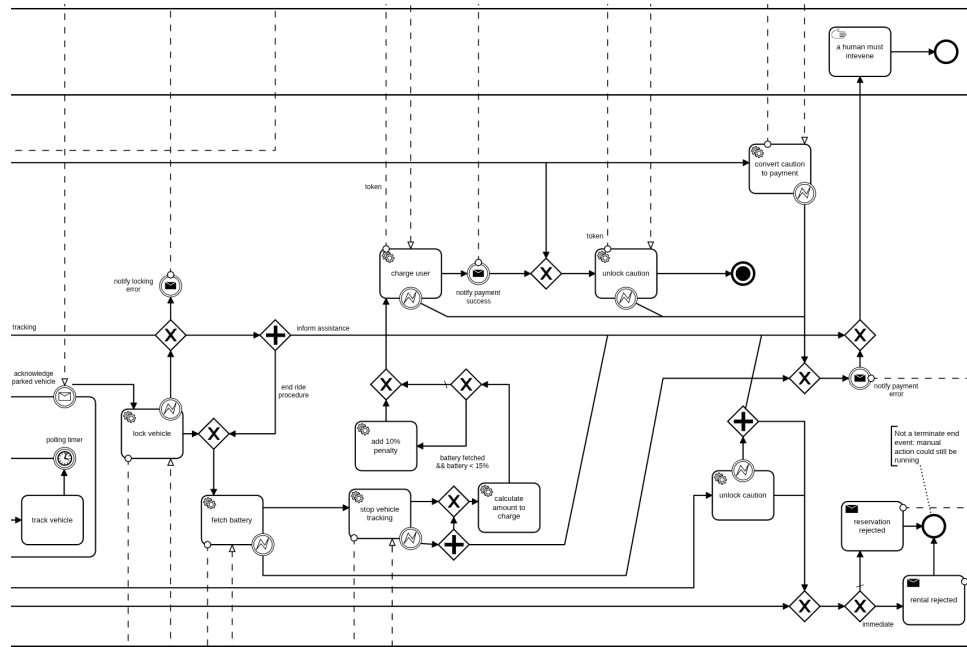


Fig. 9. BPMN documentativo per ACMEMobility, fase 3

Andando a specificare alcune note aggiuntive, si assume che ACMEMobility conosca già credenziali, dati personali e dati bancari dell'utente — necessari per richiedere gli addebiti —, che verranno raccolti dal servizio in contesti fuori dalla modellazione richiesta dal progetto (fasi di registrazione e gestione dell'account), probabilmente appoggiandosi a un servizio esterno di gestione delle carte di credito per motivi di sicurezza. Per minimizzare lo scambio di informazioni riservate sulla rete, la banca ritorna un token alla prima chiamata, che può essere riutilizzato in seguito per identificare una specifica sessione dell'utente, con la sua cauzione (eventualmente convertibile in addebito).

In ogni fase, il servizio centrale di ACMEMobility tiene traccia, nel suo database, di alcune informazioni sui veicoli (oltre a quelle sugli utenti), aggiornate al momento opportuno. I possibili 5 stati di un veicolo, infatti, includono “reserved” quando prenotato, “rented” quando in uso, “maintenance” e “charging” quando fermo per manutenzione o batteria bassa, “available” altrimenti. Per un veicolo fermo, si salva pure la stazione in cui si trova, necessario per sapere a chi inviare la richiesta di sblocco; per uno prenotato o in uso, invece, si salva l'utente. In ultimo, ACME memorizza l'ultimo valore ricevuto da FleetManagement per la batteria, appunto per poter decidere se assegnare uno stato di “charging” o “available” al momento del parcheggio. Per testare, il servizio espone un endpoint che permette di aggiornare manualmente lo stato di un veicolo (simulando, ad esempio, manutenzione o carica completata).

Se un veicolo parcheggiato non è “available”, l'applicazione dell'utente dovrebbe mostrargli un errore in fase di prenotazione o noleggio immediato. Per non complicare eccessivamente la coreografia, la nostra architettura effettua il controllo solo al momento dell'avvio della corsa (nello specifico, appena prima di avviare il tracciamento del veicolo, nello stesso *service task*). In un'implementazione reale, basterebbe espandere maggiormente i task, con opportuni casi di errore (analoghi ad altri che abbiamo definito), per effettuare il controllo sia prima della scansione immediata, sia prima di un noleggio, preferibilmente prima di aver bloccato la cauzione.

Oltre ad alcuni appena visti, la coreografia tiene conto di altri possibili casi di errore: se avvenuti prima di qualsiasi addebito (come al momento del blocco della cauzione), si invia solo un rifiuto di noleggio o prenotazione all'utente; se avvenuti dopo (come per avvio del tracking o sblocco del veicolo prima della corsa), si richiede lo sblocco della cauzione, prima di inviare una notifica. Per altri errori in fasi più critiche, si richiede l'intervento di un umano ma, allo stesso tempo, si manda sempre avanti,

in qualche modo, il processo, con l'obiettivo di non lasciare mai l'utente fermo e costretto ad aspettare — d'altra parte, la coreografia ammette attese “asincrone”, come un pagamento ritardato di qualche ora, cioè che non lo bloccano fisicamente. Ciò avviene in particolare per gli errori in fase di conclusione e pagamento (richiesta del livello della batteria oppure addebito o sblocco della cauzione), in cui si invia una notifica di errore alternativa a quella di “pagamento avvenuto con successo”, così che il processo *user* possa terminare. In caso di impossibilità di interrompere il tracking, così come per la richiesta di assistenza in fase di parcheggio, essendo un errore che non inficia in alcun modo il pagamento, si procede con calcoli e addebito e contemporaneamente si richiede l'intervento di un umano, ma senza inviare messaggi di errore.

Si noti anche che, nella modellazione, i task di errore di ACMEMobility sono seguito da *end event* senza il cerchio pieno, per indicare che non tutti i processi del pool devono essere conclusi: per quanto appena visto, altri processi potrebbero essere stati lanciati in parallelo ed essere ancora in esecuzione.

A livello logico, nel BPMN a scopo documentativo il pool di ACMEMobility è costituito da un'unica grande lane, con la sola separazione del task di intervento manuale, relegata alla lane “human assistance”, stando ad indicare una divisione tra i task automatizzati e quelli eseguiti da operatori veri.

Come già accennato, il BPMS di ACME può essere controllato da chiamate *http* — che possiamo simulare con lo script python *user.py* — per avviare o gestire i processi. Al loro arrivo, Camunda ne converte automaticamente i parametri (nel *body* di una *POST*) in variabili globali — dunque disponibili per le implementazioni, per esempio, dei *service task*.

Tra i parametri richiesti dalla “API” di ACME, risultano particolarmente utili *isImmediate* (booleano per distinguere il tipo di noleggio) e, in fase di parcheggio, l'id della stazione (ipoteticamente rilevato dal veicolo stesso tramite comunicazione a corto raggio). Inoltre, la variabile *businessKey*, già implementata e riconosciuta automaticamente in Camunda, permette di associare le richieste a uno stesso processo: usando l'id del veicolo in ogni chiamata, le comunicazioni relative a ogni corsa saranno così aggregate. Dal *cockpit* (GUI) di Camunda, i processi saranno indicati proprio dalla loro *businessKey*.

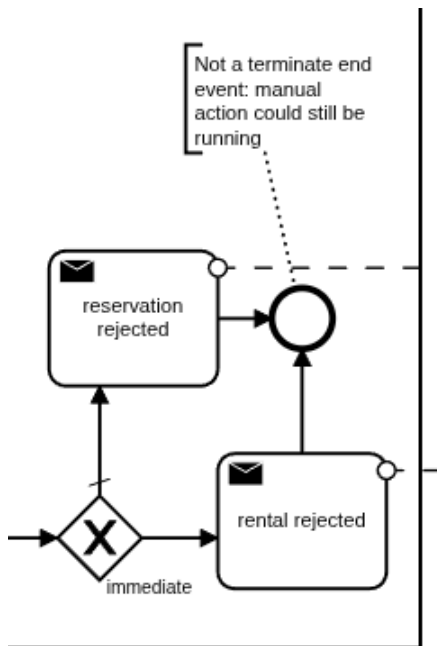


Fig. 10. BPMN documentativo per ACMEMobility, errori prima della cauzione

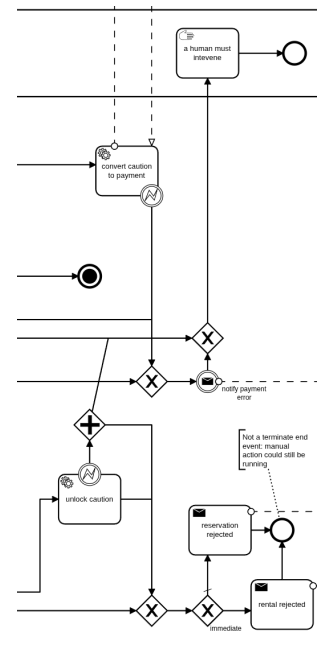


Fig. 11. BPMN documentativo per ACMEMobility, errori dopo la cauzione

3) *Prospettiva degli altri servizi:* Tutti gli altri partecipanti (banca, stazione, FleetManagement, veicolo), nel BPMN documentativo, sono modellati in maniera banale, poiché ogni loro capability viene vista fondamentalmente come un unico *service task*, avviato da una richiesta di ACME e che ritorna un risultato a sua volta. Le uniche eccezioni sono costituite da alcuni loop, per FleetManagement e veicolo, nella fase del **tragitto**. Infatti, quando ACMEMobility le richiede l'inizio del tracking, FleetManagement si mette in ascolto, in attesa di informazioni, fino all'interruzione del tracking; allo stesso tempo, il veicolo noleggiato avrà ricevuto, dallo stesso FleetManagement, una richiesta di iniziare a condividere il suo stato periodicamente (con una *POST* via *http*), anch'esso fino a una notifica di terminazione. Le query di ACME sono invece gestite tramite un task separato di FleetManagement: il motivo è che i dati trasmessi dai veicoli vengono salvati in un database (di FleetManagement) e, al momento dell'interrogazione, vengono ritornati quelli più recenti.

La modellazione raffigura FleetManagement come un unico pool e un'unica lane, invece che dividere i task di condivisione batteria e condivisione posizione, sia per leggibilità (visto che i task sarebbero stati identici), sia perché, dal punto di vista di ACME, esso appare come un unico servizio (implementato con dei microservizi al suo interno).

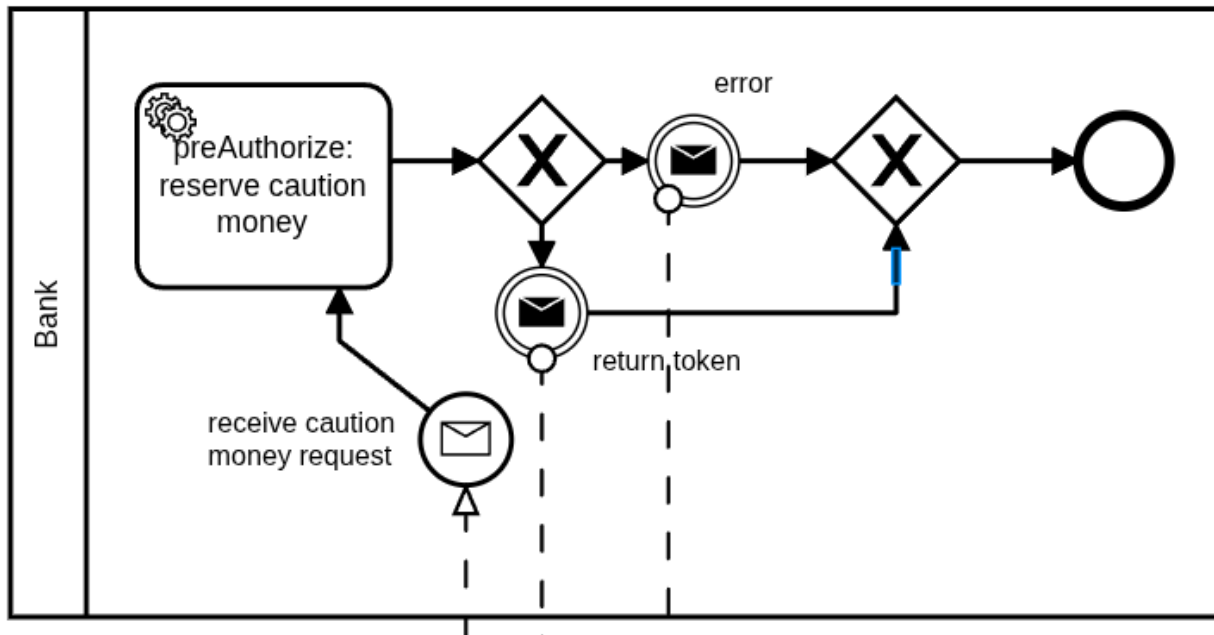


Fig. 12. BPMN documentativo per Bank (parte 1)

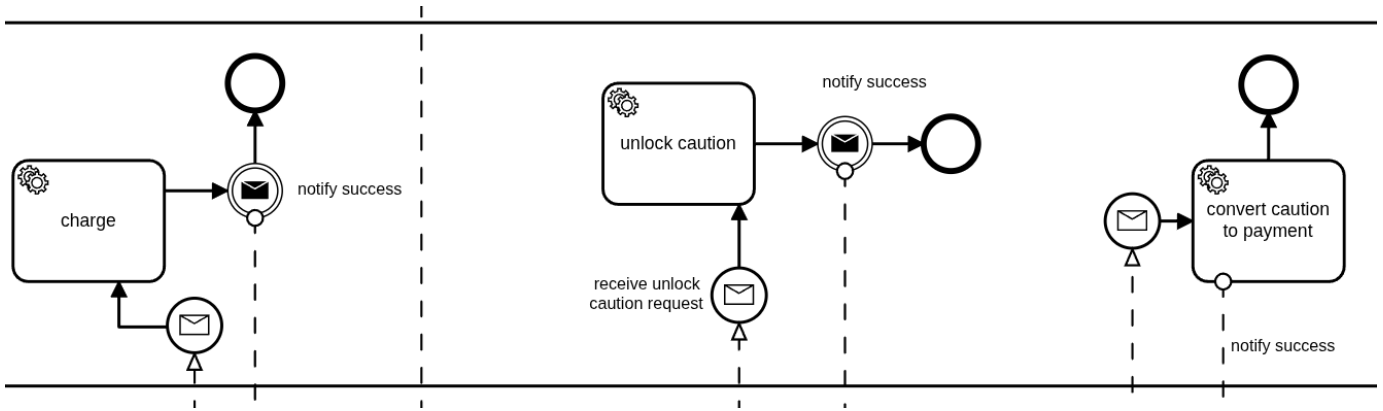


Fig. 13. BPMN documentativo per Bank (parte 2)

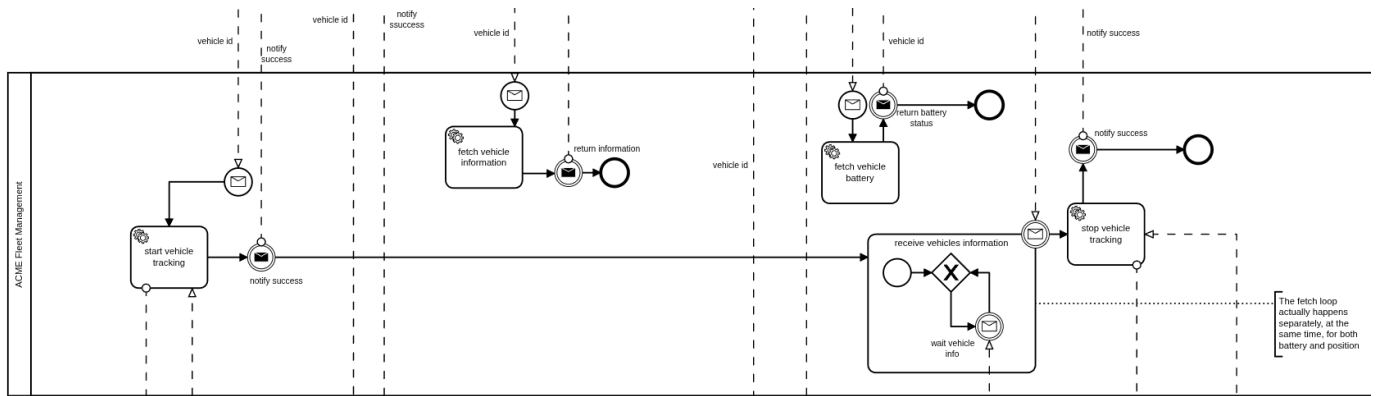


Fig. 14. BPMN documentativo per FleetManagement

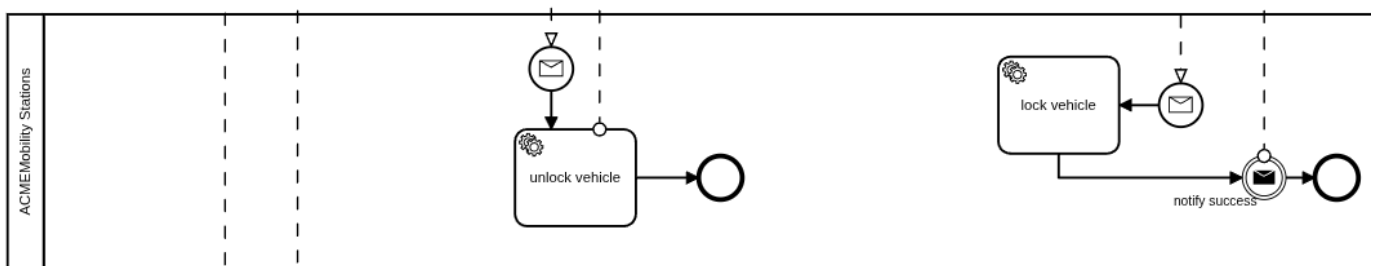


Fig. 15. BPMN documentativo per una Stazione

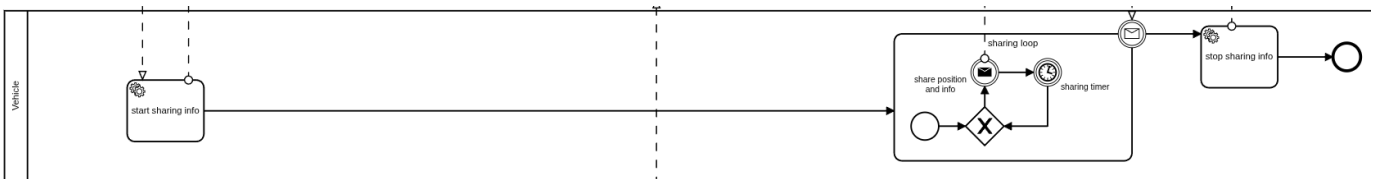


Fig. 16. BPMN documentativo per un Veicolo

## B. I servizi

I servizi sono stati realizzati tenendo in mente un’architettura in cui ognuno sia agnostico riguardo all’implementazione degli altri, comunicando tramite API documentate (con protocolli *http* e *SOAP*). Perciò, eventuali dati non-volatili di ogni servizio — e solo quelli necessari — sono memorizzati su un proprio database, accessibile solo da esso e non dall’esterno. Nell’implementazione, essendo più che altro a scopo dimostrativo, le basi di dati sono in realtà solo simulate (tramite file, classi o pacchetti) e accedute tramite opportune interfacce (chiamate di funzione), piuttosto che implementate come veri e propri microservizi a loro volta, interconnessi con gli altri.

Principalmente, i dati memorizzati stabilmente (tutti documentati in *docs/implementation/db.md*) riguardano le liste di ogni utente, stazione e veicolo, con informazioni aggiornate su ognuno dopo ciascuna operazione (come la stazione in cui si trova parcheggiato, oppure l’utente che l’ha noleggiato). La cronologia del livello di batteria e dei movimenti dei mezzi sono invece mantenute dai rispettivi sotto-servizi di FleetManagement: il core di ACMEMobility li richiede in tempo reale e non ha al bisogno di salvarli più a portata di mano, al momento.

All’avvio dell’architettura con *docker-compose*, i database vengono riempiti con alcuni dati di esempio, per identificativi e informazioni su utenti, stazioni e veicoli.

Inoltre, abbiamo cercato di delegare il meno possibile al lato *user*, essendo i *frontend* delle applicazioni da ritenersi sempre inaffidabili. La “API” di ACMEMobility richiede meno informazioni possibili all’utente; per esempio, la stazione in cui si trova un veicolo viene già memorizzata e aggiornata costantemente, mentre quella in cui verrà parcheggiato a fine corsa assumiamo che venga “letta” tramite comunicazione a breve raggio (in fase di inserimento nel terminale) e comunicata in modo sicuro.

## V. DIAGRAMMI

In questa sezione discutiamo maggiormente la produzione dei diagrammi, oltre ai risultati sulla *connectedness*.

Per produrre il diagramma UML (con profilo TinySOA) della Service Architecture a partire dal BPMN, abbiamo seguito il metodo SOMA (Service-Oriented Modeling and Architecture). Prima di tutto, ogni pool rappresenta una *capability*, modellata come un’*Entity*. Ogni messaggio scambiato tra i partecipanti indica un’operazione esposta dalla *Service Interface* del servizio/entità che riceve il messaggio — per esempio, ACMEMobility permette di avviare un noleggio.

Poiché il partecipante ACMEMobility costituisce il fulcro e l’orchestratore dei processi, esso è stato rappresentato come un *Task Service*, che può usare o invocare gli altri (micro-) servizi, come dipendenze, con l’elemento *use* di UML — interagendoci con protocolli quali *http* e *SOAP*.

L’utente e i veicoli, essendo entità fisiche (umane o IoT), sono stati rappresentati come semplici attori esterni che interagiscono con il sistema, in contrapposizione alle *Entity*, dei veri servizi, con delle *capability*, che agiscono ognuno su un dominio specifico. Inoltre, le stazioni, essendo tante ma implementate allo stesso modo, sono rappresentate come una *box* invece che piatte.

Il risultante diagramma UML/TinySOA, dunque, combina le sintassi di *Class Diagram* (con servizi e interfacce) e *Component Diagram* (con più dettagli sulle entità) per mostrare, in un unico grafico, l’architettura del sistema su più livelli contemporaneamente.

### A. Coreografia con sintassi formale

**Nota** — Gli operatori di scelta (+) e di composizione parallela (|) hanno minore precedenza rispetto alla composizione sequenziale (;).

Si considerino i seguenti partecipanti:

- **U:** User
- **A:** ACMEMobility
- **B:** Bank
- **F:** Fleet Management
- **S:** Station
- **V:** Vehicle

La coreografia  $C$  si definisce come la scelta tra un noleggio immediato ( $C_{imm}$ ) e una prenotazione ( $C_{res}$ ):

$$C = C_{imm} + C_{res}$$

### Noleggio immediato:

$$C_{imm} =$$

$$\begin{aligned} & req\_imm : U \rightarrow A; \\ & caut\_lock : A \rightarrow B; ( \\ & \quad caut\_lock\_err : B \rightarrow A; C_{err\_reject} + \\ & \quad caut\_locked : B \rightarrow A; notify\_imm : A \rightarrow U; C_{ride\_start} \\ & ) \end{aligned}$$

### Prenotazione:

$$C_{res} =$$

$$\begin{aligned} & req\_res : U \rightarrow A; \\ & caut\_lock : A \rightarrow B; ( \\ & \quad caut\_lock\_err : B \rightarrow A; C_{err\_reject} + \\ & \quad caut\_locked : B \rightarrow A; reserved : A \rightarrow U; (C_{cancel} + C_{scan\_res}) \\ & ) \end{aligned}$$

**Nota** — Ogni richiesta alla banca può ritornare un *ok* o un *errore*, nel qual caso si va a *err\_manual*.

$$C_{cancel} =$$

$$\begin{aligned} & cancel : U \rightarrow A; ( \\ & \quad caut\_release : A \rightarrow B; (caut\_release\_err : B \rightarrow A; C_{err\_manual} + caut\_released : B \rightarrow A) + \\ & \quad charge\_pen : A \rightarrow B; (charge\_pen\_err : B \rightarrow A; C_{err\_manual} + charged\_pen : B \rightarrow A) \\ & ) \end{aligned}$$

$$C_{scan\_res} = scan\_res : U \rightarrow A; C_{ride\_start}$$

### Viaggio e inizio e fine del noleggio:

**Nota** — Anche il veicolo potrebbe ritornare successo o errore: per semplicità, qui viene mostrato come se solo FleetManagement possa ritornare un errore. Molti possibili errori come questo si potrebbero aggiungere in maniera analoga ad altri presenti, ma complicherebbero le definizioni, aggiungendo tante scelte e tanti livelli di annidamento.

$$\begin{aligned}
C_{ride\_start} = & \\
& track\_start : A \rightarrow F; share\_start : F \rightarrow V; share\_started : V \rightarrow F; \\
& (track\_err : F \rightarrow A; C_{err\_release} + \\
& \quad track\_started : F \rightarrow A; unlock : A \rightarrow S; \\
& \quad (unlock\_err : S \rightarrow A; C_{err\_release} + unlock\_ok : S \rightarrow A; unlocked : A \rightarrow U; C_{ride}) \\
& )
\end{aligned}$$

**Nota** — Qui omettiamo per semplicità il loop in cui FleetManagement riceve periodicamente dati (posizione e batteria) dal veicolo. Questo sarebbe un ciclo separato che avviene in parallelo, dato che le informazioni vengono memorizzate (in un database) per poter essere ritornate ad ACMEMobility su richiesta, ma le sue formule sarebbero banalmente analoghe all'altro loop di FleetManagement (ripetizione 0 o più volte di condivisione dati e risposta di conferma).

**Nota** — Si tenga presente che un tentativo senza successo di parcheggio e blocco del veicolo può avvenire 1 o più volte, fino a un successo o a una richiesta di assistenza: da coreografia, deve avvenire almeno una volta, perché l'utente possa richiedere l'assistenza.

$$\begin{aligned}
C_{ride} = & \\
& (pos\_fetch : A \rightarrow F; pos\_fetched : F \rightarrow A)^*; \\
& (C_{park\_attempt})^*; ( \\
& \quad C_{park\_attempt}; assist : U \rightarrow A + \\
& \quad park : U \rightarrow A; lock : A \rightarrow S; lock\_ok : S \rightarrow A; C_{ride\_end} \\
& ) \\
C_{park\_attempt} = & (park : U \rightarrow A; lock : A \rightarrow S; lock\_err : S \rightarrow A; park\_err : A \rightarrow U)
\end{aligned}$$

**Nota** — Ogni richiesta alla banca può ritornare *ok* o *error*, nel qual caso si va a *err\_manual*.

**Nota** — Se fallisce la richiesta *track\_stop* (*err\_track\_stop*), si va avanti, ma verrà richiesto un intervento manuale in seguito.

**Nota** — In questa parte conclusiva, nella maggior parte dei casi di errore si usa *notify\_err\_manual*, cioè si notifica l'utente, poiché, nella coreografia, egli è in attesa o di un messaggio di fine corsa (con successo, per esempio con una ricevuta) o di un messaggio di errore. Se l'errore avviene al rilascio della cauzione, anche la coreografia non mostra notifiche all'utente: si assume che il problema verrà risolto da un operatore al più presto, senza però bloccare il processo.

$$\begin{aligned}
C_{ride\_end} = & \\
& ask\_batt : A \rightarrow F; ( \\
& \quad (err\_share\_batt : F \rightarrow A; C_{err\_manual\_notify}) + \\
& \quad share\_batt : F \rightarrow A; \\
& \quad track\_stop : A \rightarrow F; share\_stop : F \rightarrow V, share\_stopped : V \rightarrow F; \\
& \quad (track\_stopped : F \rightarrow A + (err\_track\_stopped : F \rightarrow A \mid C_{err\_manual})); \\
& \quad charge : A \rightarrow B; ( \\
& \quad \quad charge\_err : B \rightarrow A; C_{err\_manual\_notify} + \\
& \quad \quad charged : B \rightarrow A; receipt : A \rightarrow U; caut\_release : A \rightarrow B; ( \\
& \quad \quad \quad caut\_release\_err : B \rightarrow A; C_{err\_manual} + \\
& \quad \quad \quad caut\_released : B \rightarrow A \\
& \quad \quad ) \\
& \quad ) \\
& )
\end{aligned}$$

### Errori:

Intervento manuale:

$$C_{err\_manual} = 1$$

Intervento manuale, notificando l'utente:

$$C_{err\_manual\_notify} = notify\_err\_manual : A \rightarrow U$$

Non potendo iniziare la corsa, sblocca la cauzione e notifica l'utente, o richiedi un intervento manuale anche qui, in caso di un errore della banca:

$$\begin{aligned}
C_{err\_release} = & \\
& caut\_release : A \rightarrow B; ( \\
& \quad C_{err\_manual} + caut\_released : B \rightarrow A; C_{err\_reject} \\
& )
\end{aligned}$$

Rifiuta il noleggio subito, senza neanche coinvolgere la banca:

$$C_{err\_reject} = notify\_reject : A \rightarrow U$$

## B. Proiezioni

Le proiezioni mostrano come la formula dell'intera coreografia si applica dal punto di vista di un singolo partecipante.

Tutte le interazioni sono sincrone: la coreografia non ha ramificazioni parallele. Nel BPMN, gli unici rami paralleli portano, come secondo percorso, all'intervento manuale, che è molto semplice e non prosegue con ulteriori comunicazioni con altri servizi.

**Nota** — Usiamo degli 1 come placeholder, per indicare delle azioni che non verrebbero codificate nella proiezione in questione, non comprendendo una comunicazione tra i partecipanti — per esempio, per gli errori manuali, indicando che qualcosa succederà in quella sezione.

**Nota** — Per alcuni partecipanti, l'intera coreografia o un suo suffisso potrebbe non venire eseguito, in caso di errori occorsi nel frattempo.

**Nota** — Si ricorda che la sintassi  $proj(C, B) = \overline{operation}@A$  indica la proiezione della coreografia  $C$  sul partecipante  $B$ , contenente l'operazione  $operation$  con output verso il partecipante  $A$ , poiché c'è una linea sopra; al contrario,  $operation@A$  indica un'operazione con input ricevuto da  $A$ .

### 1) Proiezione per Bank (B):

$$proj(C_{imm}, B) = \text{caut\_lock}@A; (\overline{\text{caut\_lock\_err}}@A + \overline{\text{caut\_locked}}@A; proj(C_{ride\_start}, B))$$

$$proj(C_{res}, B) =$$

$$\begin{aligned} & \text{caut\_lock}@A; ( \\ & \quad \overline{\text{caut\_lock\_err}}@A + \\ & \quad \overline{\text{caut\_locked}}@A; (proj(C_{cancel}, B) + proj(C_{ride\_start}, B)) \\ & ) \end{aligned}$$

$$proj(C_{cancel}, B) =$$

$$\begin{aligned} & \text{caut\_release}@A; ( \\ & \quad (\overline{\text{caut\_release\_err}}@A + \overline{\text{caut\_released}}@A) + \\ & \quad \text{charge\_pen}@A; (\overline{\text{charge\_pen\_err}}@A + \overline{\text{charged\_pen}}@A) \\ & ) \end{aligned}$$

$$proj(C_{ride\_start}, B) = \text{caut\_release}@A; (\overline{\text{caut\_release\_err}}@A + \overline{\text{caut\_released}}@A; proj(C_{ride\_end}, B))$$

$$proj(C_{ride\_end}, B) =$$

$$\begin{aligned} & \text{charge}@A; ( \\ & \quad \overline{\text{charge\_err}}@A + \\ & \quad (\overline{\text{charged}}@A; \text{caut\_release}@A; (\overline{\text{caut\_release\_err}}@A + \overline{\text{caut\_released}}@A)) \\ & ) \end{aligned}$$

2) *Proiezione per Fleet Management (F):*

$$\begin{aligned}
 proj(C, F) = & ( \\
 & 1 + \\
 & \overline{track\_start@A; share\_start@V; share\_started@V}; \\
 & (\overline{track\_err@A + track\_started@A}; proj(C_{ride}, F)) \\
 & ) \\
 proj(C_{ride}, F) = & (\overline{pos\_fetch@A; pos\_fetched@A}^*; proj(C_{ride\_end}, F) \\
 proj(C_{ride\_end}, F) = & ask\_batt@A; ( \\
 & \overline{err\_share\_batt@A +} \\
 & \overline{share\_batt@A; track\_stop@A}; \\
 & \overline{share\_stop@V; share\_stopped@V}; (\overline{err\_track\_stopped@A + track\_stopped@A}) \\
 & )
 \end{aligned}$$

### 3) Proiezione per ACMEMobility (A):

$$\begin{aligned}
proj(C_{imm}, A) &= req\_imm@U; \overline{caut\_lock}@B; \\
&\quad (caut\_lock\_err@B; \overline{notify\_reject}@U + caut\_locked@B; proj(C_{ride\_start}, A)) \\
proj(C_{res}, A) &= req\_res@U; \overline{caut\_lock}@B; ( \\
&\quad caut\_lock\_err@B; \overline{notify\_reject}@U + \\
&\quad caut\_locked@B; \overline{notify\_reserved}@U; (proj(C_{cancel}, A) + proj(C_{scan\_res}, A)) \\
&\quad ) \\
proj(C_{cancel}, A) &= cancel@U; ( \\
&\quad \overline{caut\_release}@B; (caut\_release\_err@B + caut\_released@B) + \\
&\quad \overline{charge\_pen}@B; (charge\_pen\_err@B + charged\_pen@B) \\
&\quad ) \\
proj(C_{scan\_res}, A) &= scan\_res@U; proj(C_{ride\_start}, A) \\
proj(C_{ride\_start}, A) &= \overline{track\_start}@F; ( \\
&\quad track\_err@F; proj(C_{err\_release}, A) + \\
&\quad track\_started@F; \overline{unlock}@S; ( \\
&\quad \quad unlock\_err@S; proj(C_{err\_release}, A) + \\
&\quad \quad unlock\_ok@S; \overline{unlocked}@U; proj(C_{ride}, A) \\
&\quad ) \\
&\quad ) \\
proj(C_{ride}, A) &= \overline{(pos\_fetch@F; pos\_fetched@F)^*}; proj(C_{park\_attempt}^*, A); ( \\
&\quad proj(C_{park\_attempt}, A); assist@U + \\
&\quad park@U; \overline{lock}@S; lock\_ok@S; proj(C_{ride\_end}, A) \\
&\quad ) \\
proj(C_{park\_attempt}, A) &= park@U; \overline{lock}@S; lock\_err@S; \overline{park\_err}@U \\
proj(C_{ride\_end}, A) &= \overline{ask\_batt}@F; ( \\
&\quad err\_share\_batt@F; \overline{notify\_err\_manual}@U + \\
&\quad \overline{share\_batt}@F; \overline{track\_stop}@F; ((err\_track\_stopped@F \mid 1) + track\_stopped@F); \\
&\quad \overline{charge}@B; ( \\
&\quad \quad charge\_err@B; \overline{notify\_err\_manual}@U + \\
&\quad \quad charged@B; \overline{receipt}@U; \\
&\quad \quad \overline{caut\_release}@B; ((caut\_release\_err@B; 1) + caut\_released@B) \\
&\quad ) \\
&\quad ) \\
proj(C_{err\_release}, A) &= \overline{caut\_release}@B; \\
&\quad (caut\_release\_err@B; 1 + caut\_released@B); \overline{notify\_reject}@U
\end{aligned}$$

4) Proiezione per Station (S): Si noti che la proiezione per la stazione include solo lo sblocco e il loop di bloccaggio.

$$proj(C, S) = ($$

$$1 +$$

$$unlock@A; ($$

$$\overline{unlock\_err}@A +$$

$$\overline{unlock\_ok}@A;$$

$$(lock@A; \overline{lock\_err}@A)^*; (lock@A; \overline{lock\_ok}@A \mid 1)$$

$$)$$

$$)$$

5) *Proiezione per User (U):*

$$proj(C_{imm}, U) = \overline{req\_imm}@A; proj(C_{scan}, U)$$

$$proj(C_{res}, U) = \overline{req\_res}@A; ($$

$$notify\_reject@A +$$

$$notify\_reserved@A; (\overline{cancel}@A + \overline{scan}@A; proj(C_{scan}, U))$$

$$)$$

La proiezione *scan* è valida sia per la scansione immediata che per quella dopo una prenotazione:

$$proj(C_{scan}, U) = notify\_reject@A + unlocked@A; proj(C_{ride}, U)$$

$$proj(C_{ride}, U) = (\overline{park}@A; \overline{park\_err}@A)^*; \overline{park}@A; ($$

$$park\_err@A; \overline{assist}@A +$$

$$park\_ok@A; (notify\_err\_manual@A + receipt@A)$$

$$)$$

6) *Proiezione per Vehicle (V):*

$$proj(C, V) = ($$

$$1 +$$

$$share\_start@F; \overline{share\_started}@F;$$

$$(1 + share\_stop@F; \overline{share\_stopped}@F)$$

$$)$$

Come per la definizione formale, sono stati omessi alcuni errori e il loop di condivisione della posizione del veicolo.

Un esempio di loop di interazione tra FleetManagement e Vehicle sarebbe:

$$proj(C_{ride\_loop}, V) = (pos\_fetch@F; \overline{pos\_fetched}@F)^*$$

### C. Analisi di *connectedness*

Definiamo connessa una coreografia se le sequenze di interazioni garantiscono una “sicurezza causale” (o “causality safety”), ovvero se, in ogni momento, un partecipante sa se sia il suo turno di agire, senza un controllo centrale che glielo comunichi di volta in volta.

Per l’analisi, possiamo considerare la nostra coreografia sincrona, dal momento che ogni partecipante effettua una comunicazione in seguito ad un’altra, oppure rimane ad aspettare il suo turno.

In realtà, la nostra architettura includerebbe due situazioni asincrone. La prima è costituita dalle composizioni parallele; tuttavia, queste avvengono solo all’estrema fine del processo e il secondo *branch* è sempre molto semplice, costituito dalla gestione manuale dell’errore ed eventualmente da una comunicazione conclusiva di errore all’utente: dunque, la creazione di un sotto-processo parallelo non rischia mai di interferire con il flusso delle altre comunicazioni sincrone. La seconda eccezione è costituita dal loop che avviene in background per la comunicazione delle informazioni di un veicolo verso FleetManagement, che, come specificato più volte, abbiamo ommesso dalle modellazioni per semplicità, essendo banale. Difatti, esso comporta operazioni completamente separate da quelle della coreografia, che non la intaccheranno in alcun modo: FleetManagement continua solo ad accumulare dati nel suo database; a una richiesta di ACMEMobility, ritornerà gli ultimi disponibili.

Considerando quindi la nostra coreografia sincrona, si può verificare la *connectedness* in quanto, per ogni sequenza  $o : a \rightarrow b; o' : c \rightarrow d$ , è soddisfatta la condizione  $a = c \vee a = d \vee b = c \vee b = d$ . Intuitivamente, ciò comporta che, in ogni coppia di comunicazioni consecutive tra due servizi, almeno uno è coinvolto in entrambi: si può avere una domanda e risposta tra gli stessi due servizi ( $a = d \wedge b = c$ ), o una serie di comunicazioni nella stessa direzione ( $a = c$ ), oppure un servizio che fa da ponte, ricevendo da uno e ritrasmettendo a un altro ( $b = c$ ).

In generale, quasi ogni passo del processo può essere ricondotto a una topologia a stella in cui ACMEMobility agisce da orchestratore centrale: è presente in quasi ogni comunicazione (come mittente o destinatario), facendo da ponte tra l’utente e i vari microservizi che usa come dipendenze per ambiti specifici. Esempi in cui non compare per qualche passaggio sono invece catene come quella in  $C_{ride\_start}$ , in cui la “parola” passa, in ordine, per ACME, FleetManagement, Vehicle, FleetManagement e nuovamente ad ACME: ognuno trasmette dopo aver ricevuto una comunicazione da qualcun altro.

Per garantire la *connectedness*, ci siamo accertati quindi (eventualmente correggendo anche errori precedenti) che ogni servizio producesse una risposta, come se dovesse passare il “testimone” del processo. È per questo motivo che sia la coreografia che la definizione formale (e le proiezioni) contengono molti messaggi di *ok* o di *errore*, altrimenti non si comprenderebbe come possa un servizio sapere l’esito della sua richiesta e, dunque, come procedere. Alcuni messaggi di conferma mancano solo alla fine del processo per un partecipante, per semplicità ma senza intaccare la *connectedness*: ad esempio, l’utente non aspetta una conferma dopo la richiesta di cancellazione della prenotazione; se ACMEMobility avrà problemi con l’addebito, provvederà in maniera asincrona con un’assistenza manuale.

La *connectedness*, in conclusione, fa sì che il processo possa sempre andare avanti (*liveness*) attraverso la semplice interazione tra i partecipanti e senza un controllore centrale, fino a uno dei suoi stati di terminazione.

## REFERENCES